

Objetivo

- conexión STOMP
- suscripción al tópico
- envío de eventos cuando haces click

Este código envía el punto al backend

```
if (stompRef.current?.connected) {  
  stompRef.current.publish({  
    destination: '/app/draw',  
    body: JSON.stringify(payload)  
  })  
}
```

Y se suscribe a

```
export function subscribeBlueprint(client, author, name, onMsg) {  
  return client.subscribe(`/topic/blueprints.${author}.${name}`, (m) => {  
    onMsg(JSON.parse(m.body))  
  })  
}
```

Esto significa que cada plano tiene su propio canal.

Y para el back se copian las clases de ejemplo y se hacen pequeñas modificaciones

```

@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint(...paths: "/ws-blueprints")
            .setAllowedOriginPatterns(...originPatterns: "*");
        // .withSockJS(); // optional
    }

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.enableSimpleBroker(...destinationPrefixes: "/topic", "/queue");
        registry.setApplicationDestinationPrefixes(...prefixes: "/app");
        registry.setUserDestinationPrefix(destinationPrefix: "/user");
    }
}

```

```

public record BlueprintUpdate(
    String author,
    String name,
    List<Point> points
) {}

```

```

package co.edu.eci.blueprints.dto;

import co.edu.eci.blueprints.model.Point;

public record DrawEvent(
    String author,
    String name,
    Point point
) {}

```

```

@Controller
public class BlueprintRealtimeController {

    private final SimpMessagingTemplate template;
    private final BlueprintsServices services;

    public BlueprintRealtimeController(
        SimpMessagingTemplate template,
        BlueprintsServices services) {

        this.template = template;
        this.services = services;
    }

    @MessageMapping("/draw")
    public void onDraw(DrawEvent evt) {

        try {

            Blueprint bp = services.getBlueprint(evt.author(), evt.name());
            bp.addPoint(new Point(evt.point().x(), evt.point().y()));
            services.updateBlueprint(bp);
            BlueprintUpdate upd = new BlueprintUpdate(
                evt.author(),
                evt.name(),
                bp.getPoints()
            );

            template.convertAndSend(
                "/topic/blueprints." + evt.author() + "." + evt.name(),
                upd
            );

        } catch (Exception e) {
            System.err.println("Error procesando evento draw: " + e.getMessage());
        }

    }

}

```

Luego se cambio los nombres de los endpoints a:

- **CRUD** (REST):

- `GET /api/blueprints?author=:author` → lista por autor (incluye total de puntos).
- `GET /api/blueprints/:author/:name` → puntos del plano.
- `POST /api/blueprints` → crear.
- `PUT /api/blueprints/:author/:name` → actualizar.
- `DELETE /api/blueprints/:author/:name` → eliminar.

Y se configuro la seguridad para permitir el flujo de websocket

```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .cors(Customizer.withDefaults())
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(...patterns: "/actuator/health", "/api/auth/login").permitAll()
            .requestMatchers(...patterns: "/v3/api-docs/**", "/swagger-ui/**", "/swagger-ui.html")
                .permitAll()

            .requestMatchers(HttpMethod.GET, ...patterns: "/api/blueprints/**")
                .hasAuthority(authority: "SCOPE_blueprints.read")

            .requestMatchers(HttpMethod.POST, ...patterns: "/api/blueprints/**")
                .hasAuthority(authority: "SCOPE_blueprints.write")

            .requestMatchers(HttpMethod.PUT, ...patterns: "/api/blueprints/**")
                .hasAuthority(authority: "SCOPE_blueprints.write")

            // permitir websocket
            .requestMatchers(...patterns: "/ws-blueprints").permitAll()
            .requestMatchers(...patterns: "/topic/**").permitAll()
            .requestMatchers(...patterns: "/app/**").permitAll()

            .anyRequest().authenticated()
        )
        .oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()));
    return http.build();
}

```

Permitiendo usar el link ws://localhost:8080/ws-blueprints

Luego se configura las variables del front

```

nv.local
VITE_API_BASE=http://localhost:8080
VITE_STOMP_BASE=http://localhost:8080 # si usas STOMP (Spring)
VITE_USE MOCK=false

```

Comportamiento de la ejecución

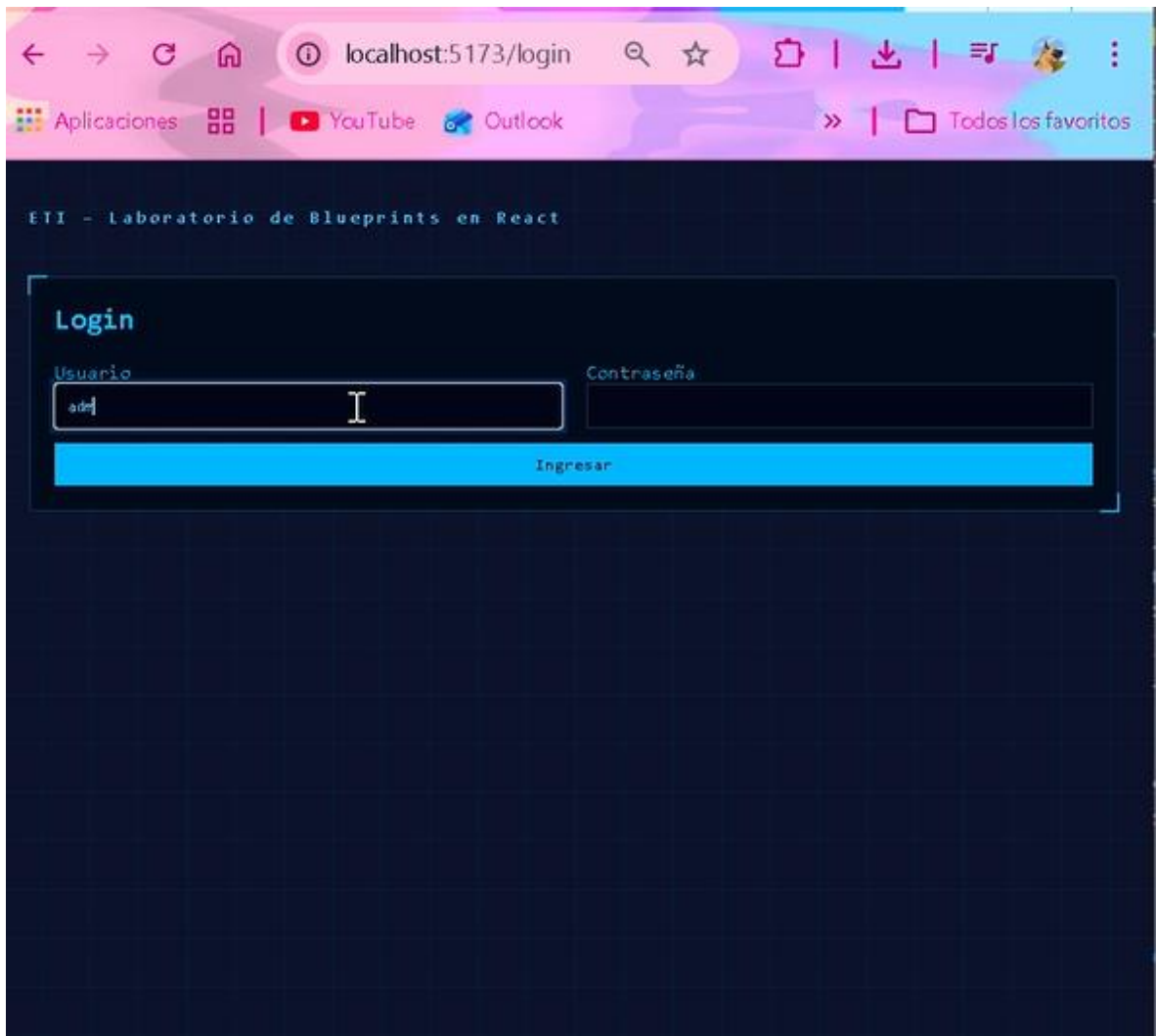
Cuando hace Onclick en el plano se envia el evento a /app/draw toma los datos y los envia al controller en @PostMapping("/draw")

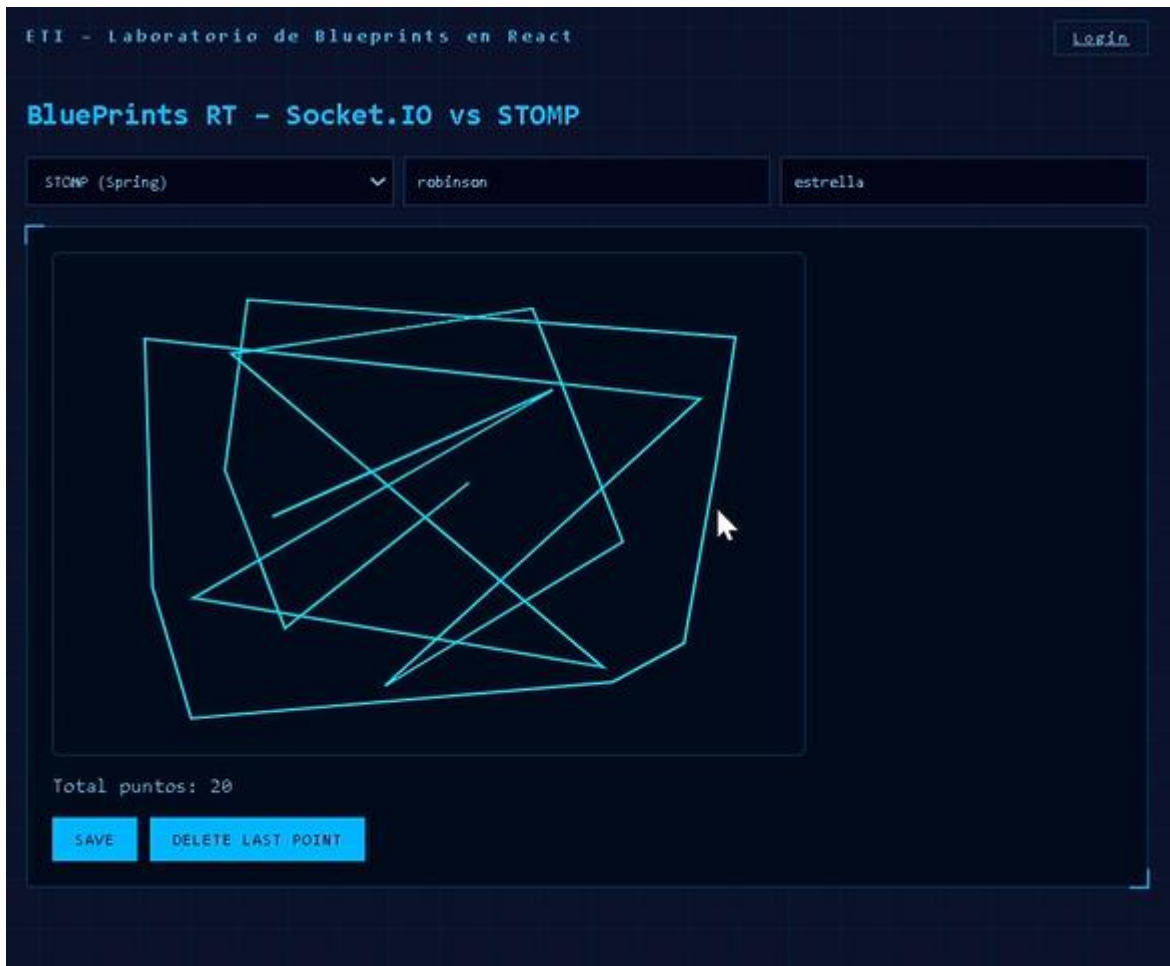
El frontend está suscrito a:

/topic/blueprints.juan.plano-1

Entonces recibe los nuevos puntos y **redibuja el canvas**.

Diseño Front





Se mejoro el front a temática oscura con azul intenso y se dividió las pantallas en login y BlueprintPage esto se decidió para dejar la app solo para la interacción entre pantallas. también se decidió crear un botón que actualice y otro que borre los datos de los puntos de cada actor

Por último, se protegió la ruta de BlueprintPage para que no se pueda usar la pantalla sin generar el token de login.



stomp.mp4

```
1 import { Navigate } from 'react-router-dom'
2
3 export default function PrivateRoute({ children }) {
4
5     const token = localStorage.getItem('token')
6
7     if (!token || token === "null" || token === "undefined") {
8         return <Navigate to="/login" replace />
9     }
10
11     return children
12 }
```

Se configuro

```
import { z } from "zod"

export const drawPayloadSchema = z.object({

  author: z.string().min(1),

  name: z.string().min(1),

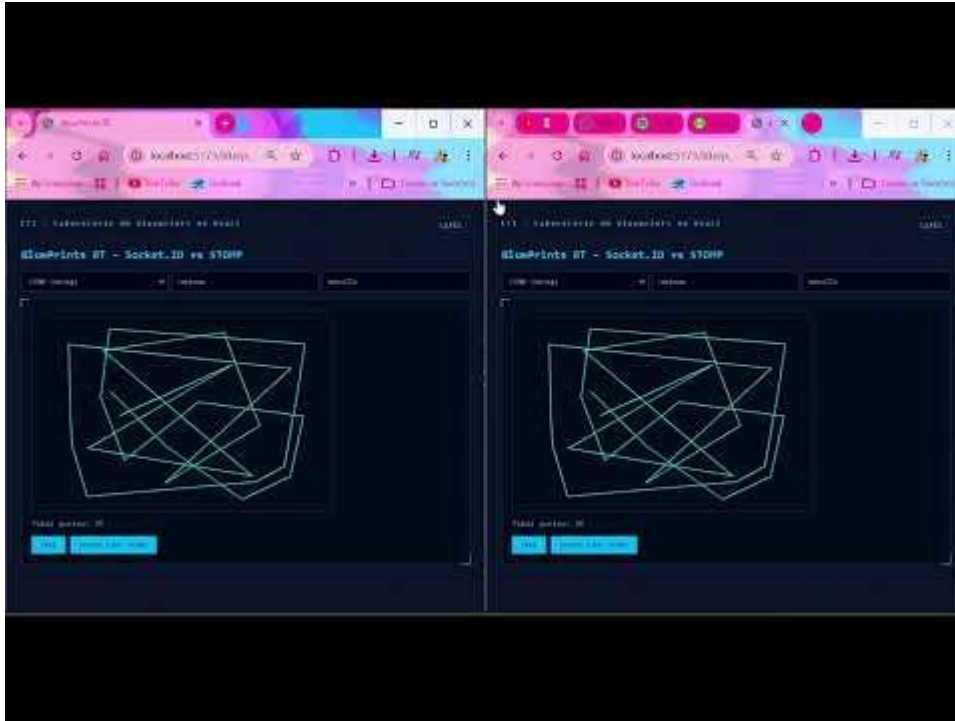
  point: z.object({
    x: z.number(),
    y: z.number()
  })
})
```

Y se uso en la pantalla principal

Se implementó validación de payloads en el cliente usando Zod. Antes de publicar eventos STOMP al endpoint /app/draw, el frontend verifica que el objeto contenga

autor, nombre del blueprint y coordenadas válidas del punto. Esto reduce errores de formato y evita enviar datos inconsistentes al backend.

Video demostrativo de colaboración vivo



https://www.youtube.com/watch?v=H2AE8_X56Eg

En este laboratorio se utilizó la licencia MIT

Comparativa

Característica: Protocolo

- Socket.IO (Node.js): Propio (con fallback a HTTP Long Polling).
- STOMP (Spring): Sub-protocolo estándar sobre WebSockets.

Característica: Facilidad

- Socket.IO (Node.js): Muy alta; abstracción simple de "salas".
- STOMP (Spring): Requiere configuración de brokers y prefijos.

Característica: Modelo

- Socket.IO (Node.js): Basado en eventos (Emit/On).
- STOMP (Spring): Basado en mensajería (Pub/Sub) tipo MQ.

Característica: Escalabilidad

- Socket.IO (Node.js): Requiere Redis Adapter para múltiples nodos.
- STOMP (Spring): Integración nativa con RabbitMQ o ActiveMQ.

Decisión del equipo: Usamos STOMP por su integración con sistemas de mensajería y mejor escalabilidad en entornos distribuidos.

MIT License

Copyright (c) 2025

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.